

Шпаргалка по Puppet

Ресурсы и манифесты, классы и модули, переменные и факты, зависимости и уведомления, шаблоны и данные
Hierarchy, команды агента и сервера, проверка и отладка.

Как это работает

- **Манифест** (.pp) описывает желаемое состояние системы, а не шаги по его достижению.
- **Ресурс** — единица описания: пакет, файл, служба, пользователь.
- **Каталог** — собранный для конкретного узла набор ресурсов; агент приводит систему к нему.
- **Факты** — сведения об узле, собранные Facter: система, адреса, память.
- **Идемпотентность**: повторный запуск ничего не меняет, если состояние уже верное.

Ресурс

```
package { 'nginx':
  ensure => installed,
}

file { ['/etc/nginx/conf.d/site.conf':
  ensure => file,
  owner   => 'root',
  group   => 'root',
  mode    => '0644',
  content => template('web/site.conf.erb'),
  require => Package['nginx'],
  notify  => Service['nginx'],
}

service { 'nginx':
  ensure => running,
  enable => true,
}
```

Тип ресурса пишется строчными буквами (package), ссылка на него — с заглавной (Package['nginx']).

Частые типы ресурсов

- package**
ensure: installed, latest, absent или конкретная версия
- file**
ensure: file, directory, link, absent; content, source, mode, owner
- service**
ensure: running или stopped; enable — автозапуск
- user / group**
Учётные записи: uid, groups, shell, home
- exec**
Команда; обязательно ограничьте unless, onlyif или creates
- cron**
Задание по расписанию: command, hour, minute
- host / mount**
Записи в /etc/hosts; точки монтирования

Порядок и уведомления

```
require => Package['nginx']
  Выполнить после указанного ресурса

before => Service['nginx']
  Выполнить до указанного

notify => Service['nginx']
  При изменении уведомить ресурс — служба перезапустится

subscribe => File['/etc/app.conf']
  Обратная сторона notify: следить за чужими изменениями

Package['nginx'] -> File['/etc/site.conf'] ~> Service['nginx']
  Цепочка стрелками: порядок и уведомление
```

Классы и модули

```
class webserver (
  Integer $port = 80,
  String  $doc_root = '/var/www',
) {
  package { 'nginx': ensure => installed }

  file { ['/etc/nginx/conf.d/app.conf':
    content => epp('webserver/app.conf.epp', {
      'port' => $port,
    }),
    notify  => Service['nginx'],
  }

  service { 'nginx':
    ensure => running,
    enable => true,
  }
}

include webserver

class { 'webserver':
  port => 8080,
}
```

Раскладка модуля: manifests/, templates/, files/, data/, spec/. Имя файла соответствует имени класса: webserver::ssl живёт в manifests/ssl.pp.

Переменные, факты, условия

```
$packages = ['curl', 'http']
    Массив; хеш — { 'ключ' => 'значение' }

$facts['os']['family']
    Факты узла: семейство системы, версия, адреса

if $facts['os']['family'] == 'Debian' { ... }
    Условие

case $facts['os']['name'] { 'Ubuntu': {...} default: {...} }
    Ветвление по значению

$pkg = $facts['os']['family'] ? { 'Debian' => 'apache2', default => 'httpd' }
    Выбор значения по условию

package { $packages: ensure => installed }
    Один ресурс для каждого элемента массива

Переменные в Puppet неизменяемы: присвоить значение второй раз в той же области нельзя.
```

Шаблоны

```
err('модуль/файл.err', { 'port' => $port })
    Шаблон на языке Puppet с явными параметрами — предпочтительный вариант

template('модуль/файл.erb')
    Шаблон ERB: переменные доступны как <%= @port %>

file { ...: source => 'puppet:///modules/web/app.conf' }
    Готовый файл из каталога files/ модуля
```

Данные: Hiera

```
# hiera.yaml
version: 5
hierarchy:
  - name: "Узел"
    path: "nodes/%{trusted.certname}.yaml"
  - name: "Семейство ОС"
    path: "os/%{facts.os.family}.yaml"
  - name: "Общее"
    path: "common.yaml"
```

```
lookup('webserver::port')
    Получить значение из данных

Автоматический поиск параметров
    Параметр класса webserver::port подхватывается из Hiera без вызова lookup

Разделяйте код и данные: логика — в модулях, значения для сред и узлов — в Hiera.
```

Команды

```
puppet apply manifest.pp
    Применить манифест локально, без сервера

puppet apply --noop manifest.pp
    Показать, что изменилось бы, ничего не делая

puppet agent -t
    Разовый запуск агента с подробным выводом

puppet agent -t --noop
    Пробный прогон против сервера

puppet parser validate файл.pp
    Проверка синтаксиса

puppet-lint манифест.pp
    Проверка стиля

factor -p os
    Посмотреть факты узла

puppet resource service nginx
    Текущее состояние ресурса в виде манифеста

puppet module install автор-модуль
    Установить модуль из Forge
```

Сервер и узлы

```
puppetserver ca list --all
    Запросы на сертификаты и выданные сертификаты

puppetserver ca sign --certname узел
    Подписать сертификат узла

puppetserver ca clean --certname узел
    Отозвать: нужно при пересоздании машины с тем же именем

/etc/puppetlabs/code/environments/production
    Каталог среды: manifests, modules, data

site.pp
    Точка входа: сопоставление узлов и классов
```

Отладка и осторожность

- Начинайте с --noop: видно намерения без изменений на узле.
- notice() и --debug помогают понять, какие значения подставились.
- Дублирование ресурса — частая ошибка: один файл не может описываться дважды; выносите его в класс.
- ехес без ограничивающего условия выполняется каждый раз и ломает идемпотентность.
- Тесты модулей — rspec-puppet, проверка на реальной системе — Litmus или запуск в контейнере.